CM08 - Functions

Function body, return, call, prototype, stack, visibility

Louis Ledoux

2026-07-05

Function Body and Return

Function Body and Return

Live pacing: about 8 min

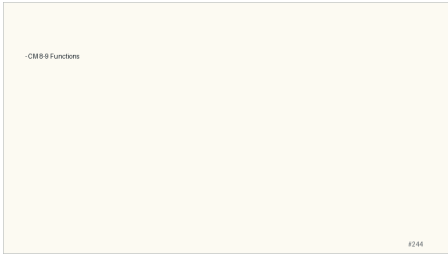
Question. What moves during a function call?

Core claim. C passes values; changing a caller object requires passing an address to that object.

Target capability. Students can explain stack frames, prototypes, array parameters, and `argc`/`argv`.

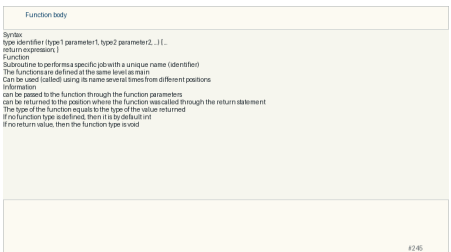
Main Path

- Start from the question: What moves during a function call?
- Build the model: C passes values; changing a caller object requires passing an address to that object.
- End with a checkable action: Students can explain stack frames, prototypes, array parameters, and `argc`/`argv`.
- Keep only this slide on the horizontal path during the live lecture.



Worked Example

- Use this as the live trace: Compare `swap(a, b)` with `swap(&a, &b)` and make students predict the printout.
- Representative source slide: 245
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.



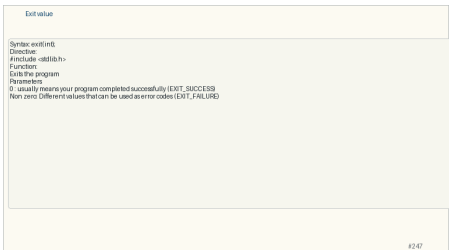
Anecdote Or Motivation

Pass-by-value is simple and strict; pointers are how C asks for explicit sharing.

- Use this only if students need a reason to care.
- If the room is already following, skip down to the check question.

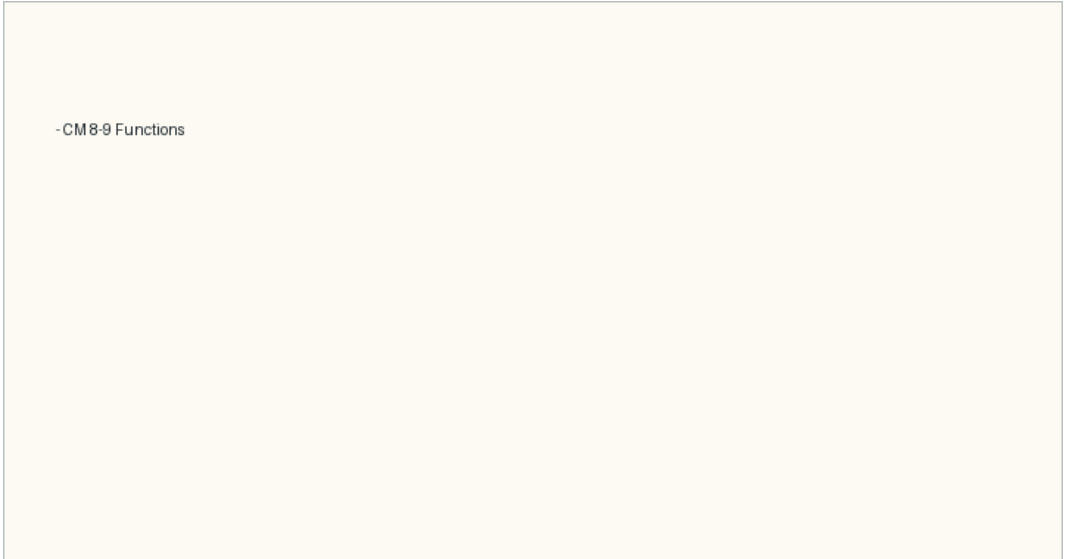
Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.



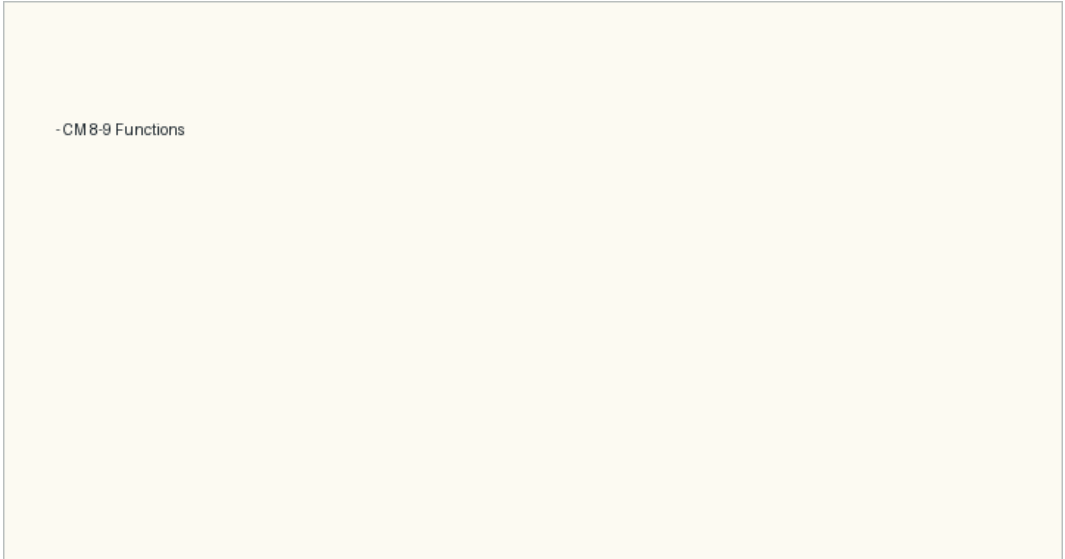
Backup Index

Original slides 244-247



Backup: CM 8-9 Functions

Original slide 244



Backup: Function body

Original slide 245

```
Syntax
type identifier (type1 parameter1, type2 parameter2,...){
return expression; }
Function
Subroutine to performs a specific job with a unique
The functions are defined at the same level as main
Can be used (called) using its name several times from different positions
Information
can be passed to the function through the function parameters
can be returned to the position where the function was called through the re
The type of the function equals to the type of the value returned
If no function type is defined, then it is by default int
If no return value, then the function type is void
```

Backup: Return value

Original slide 246

```
Syntax: return expression;
The possible types that can be returned by the eva                    on
The primitive data types
The structures
The pointers
Attention: We CANNOT return arrays! We can return a pointer char*, int*, ...
passed as argument during the function call
dynamically allocated inside the function
```

Backup: Exit value

Original slide 247

```
Syntax: exit(int);
Directive:
#include <stdlib.h>
Function:
Exits the program
Parameters
0 : usually means your program completed successfully (EXIT_SUCCESS)
Non zero: Different values that can be used as error codes (EXIT_FAILURE)
```

Function Calls and Prototypes

Function Calls and Prototypes

Live pacing: about 8 min

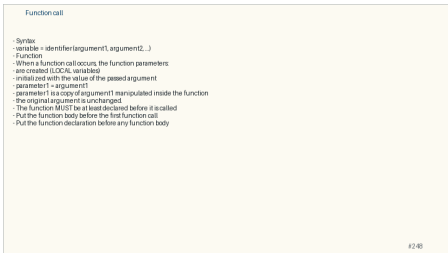
Question. What moves during a function call?

Core claim. C passes values; changing a caller object requires passing an address to that object.

Target capability. Students can explain stack frames, prototypes, array parameters, and `argc`/`argv`.

Main Path

- Start from the question: What moves during a function call?
- Build the model: C passes values; changing a caller object requires passing an address to that object.
- End with a checkable action: Students can explain stack frames, prototypes, array parameters, and `argc`/`argv`.
- Keep only this slide on the horizontal path during the live lecture.



Worked Example

- Use this as the live trace: Compare `swap(a, b)` with `swap(&a, &b)` and make students predict the printout.
- Representative source slide: 248
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

Anecdote Or Motivation

Pass-by-value is simple and strict; pointers are how C asks for explicit sharing.

- Use this only if students need a reason to care.
- If the room is already following, skip down to the check question.

Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

Backup Index

Original slides 248-251

Function call

- Syntax
- variable = identifier(argument1, argument2, ...)
- Function
- When a function call occurs, the function parameters:
- are created (LOCAL variables)
- initialized with the value of the passed argument
- parameter1 = argument1
- parameter1 is a copy of argument1 manipulated inside the function
- the original argument is unchanged.
- The function MUST be at least declared before it is called
- Put the function body before the first function call
- Put the function declaration before any function body

Backup: Function call

Original slide 248

- Syntax
- variable = identifier(argument1, argument2, ...)
- Function
- When a function call occurs, the function parameters:
- are created (LOCAL variables)
- initialized with the value of the passed argument
- parameter1 = argument1
- parameter1 is a copy of argument1 manipulated inside the function
- the original argument is unchanged.
- The function MUST be at least declared before it is called
- Put the function body before the first function call
- Put the function declaration before any function body

Backup: Function prototype

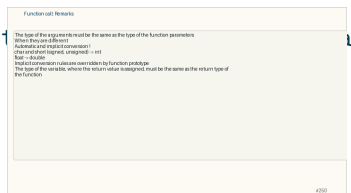
Original slide 249

- Syntax
- type identifier (type1 parameter1, type2 parameter2 ,...);
- The function declaration is the function prototype
- The complete bloc of the function is replaced by a semicolon ;
- The name of the parameters can be omitted, but not their type

Backup: Function call: Remarks

Original slide 250

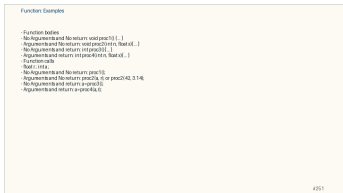
```
The type of the arguments must be the same as the t                    aram
When they are different
Automatic and implicit conversion !
char and short (signed, unsigned) -> int
float -> double
Implicit conversion rules are overridden by function prototype
The type of the variable, where the return value is assigned, must be the sa
```



Backup: Function: Examples

Original slide 251

- Function bodies
- No Arguments and No return: void proc1() { ... }
- Arguments and No return: void proc2(int n, float x){ ... }
- No Arguments and return: int proc3(){ ... }
- Arguments and return: int proc4(int n, float x){ ... }
- Function calls
- float r ; int a ;
- No Arguments and No return: proc1();
- Arguments and No return: proc2(a, r); or proc2(42, 3.14);
- No Arguments and return: a=proc3();
- Arguments and return: a=proc4(a,r);



Stack and Visibility

Stack and Visibility

Live pacing: about 8 min

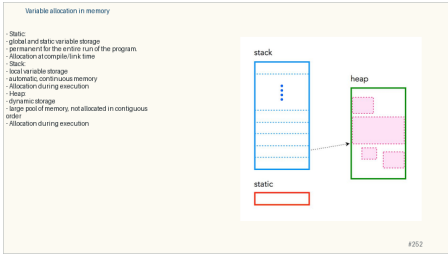
Question. What is the role of Stack and Visibility in the course story?

Core claim. Stack and Visibility is one step in the path from source text to reliable execution.

Target capability. Students can connect the topic to a concrete command, program state, or memory state.

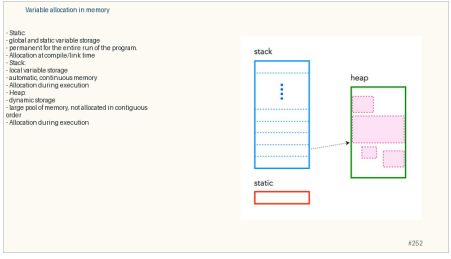
Main Path

- Start from the question: What is the role of Stack and Visibility in the course story?
- Build the model: Stack and Visibility is one step in the path from source text to reliable execution.
- End with a checkable action: Students can connect the topic to a concrete command, program state, or memory state.
- Keep only this slide on the horizontal path during the live lecture.



Worked Example

- Use this as the live trace: Use one short trace, then ask students which state changed and which state did not.
- Representative source slide: 252
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.



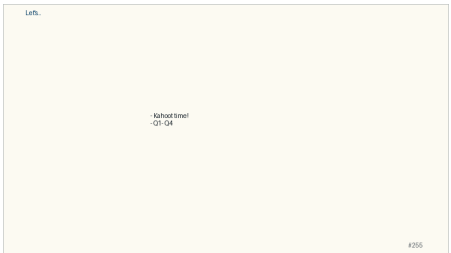
Anecdote Or Motivation

Keep the discussion anchored in observable behavior: command output, compiler message, or memory drawing.

- Use this only if students need a reason to care.
- If the room is already following, skip down to the check question.

Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

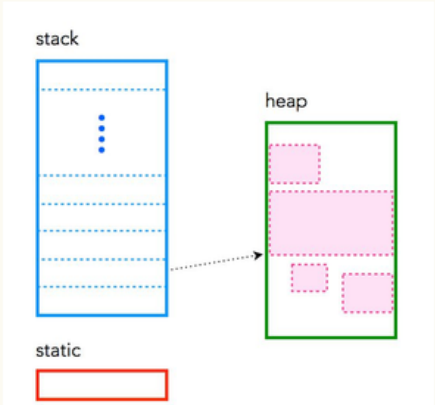


Backup Index

Original slides 252-255

Variable allocation in memory

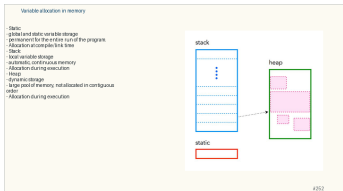
- Static:
- global and static variable storage
- permanent for the entire run of the program.
- Allocation at compile/link time
- Stack:
- local variable storage
- automatic, continuous memory
- Allocation during execution
- Heap:
- dynamic storage
- large pool of memory, not allocated in contiguous order
- Allocation during execution



Backup: Variable allocation in memory

Original slide 252

- Static:
- global and static variable storage
- permanent for the entire run of the program.
- Allocation at compile/link time
- Stack:
- local variable storage
- automatic, continuous memory
- Allocation during execution
- Heap:
- dynamic storage
- large pool of memory, not allocated in contiguous order
- Allocation during execution



Backup: Stack

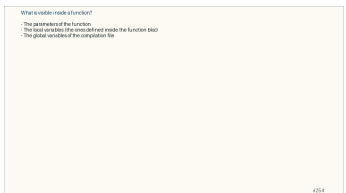
Original slide 253

- Each time a function call occurs, the stack grows to store the local variables of the function
- Each time a function returns, the stack shrinks and removes the function local variables
- So, stack variables ONLY exist while the function that created them is running
- No need to manage the memory yourself, variables are allocated and freed automatically
- The stack has size limits

Backup: What is visible inside a function?

Original slide 254

- The parameters of the function
- The local variables (the ones defined inside the function bloc)
- The global variables of the compilation file
- The imported global variables (specified by extern)
- The other functions declared before this function
- inside the compilation file
- external



Backup: Let's...

Original slide 255

- Kahoot time!
- Q1- Q4